УЧЕБНЫЙ ПЛАН

AI, LLM и агентные системы

Персональная программа для middle Data Engineer:
от первого вызова API до мультиагентной системы в закрытом контуре

Темп	5–10 часов в неделю	**Длительность**	~14–18 недель
Сквозной проект	RAG-система	**Контур**	гибрид (локально + облако)
Принцип	provider-agnostic	**Версия**	1.0 · ноябрь 2026

Содержание

1. Цели и договорённости . . . 3
2. Исходные данные: профиль и ресурсы . . . 3
3. Большая картина . . . 4
4. Ключевые принципы курса . . . 4
5. Этапы обучения . . . 5
Этап 0. Фундамент LLM и API . . . 5
Этап 1. Локальный инференс-стек . . . 5
Этап 2. CLI и агентное окружение на Windows . . . 6
Этап 3. Агенты изнутри . . . 7
Этап 4. RAG с нуля на локальных моделях . . . 8
Этап 5. Скиллы и MCP . . . 9
Этап 6. Spec-driven development . . . 9
Этап 7. Мультиагентные системы . . . 10
Этап 8. Закрытый контур и продакшн . . . 11
6. Must-have стек на конец 2026 года . . . 11
7. Карточка терминов . . . 12
8. Формат работы . . . 12

1. Цели и договорённости

Цель: освоить AI, LLM и агентные системы, чтобы расширить кругозор и продвинуться на работе — уметь автоматизировать и выполнять более сложные комплексные задачи.

Критически важное требование: provider-agnostic архитектура. Уметь в зависимости от ситуации подключать **любой LLM** — локальный или мощный облачный через API — и одинаково уверенно работать как в открытом, так и в закрытом (air-gapped) контуре.

- **Стиль:** преподавательский — подробно, с нюансами и примерами, на русском.
- **Ритм:** по одному этапу за раз, теория + мини-практика в каждом шаге.
- **Сквозной проект:** RAG-система. Text-to-SQL агент и агент качества данных — мини-упражнения на Этапе 7.
- **Контур:** гибрид. Основа — локальные модели; в ключевых точках — сравнение с облачными фронтирными моделями («локальная 14B ошиблась, облачная справилась — разберём почему»). Гибрид включает закрытый контур как частный случай.

2. Исходные данные: профиль и ресурсы

Уровень: middle Data Engineer; уверенная работа с данными. Python, SQL, Docker — уже база, время на неё не тратим.

Оборудование

ОС	Windows (+ WSL2 для агентного окружения)
RAM	64 GB
CPU	Intel Core i5-14600K
GPU	GeForce RTX 5060 Ti, **16 GB VRAM**

Что это значит на практике: локально комфортно работают модели класса **14B в квантизации Q4–Q6** (Qwen, Hermes и др.) и впритык ~20B (например, gpt-oss-20b в MXFP4). Модели 32B+ — на пределе в сильной квантизации. Это не ограничение, а учебный полигон: на своём железе изучаем трейд-оффы «качество / память / скорость» — ежедневную реальность закрытых контуров.

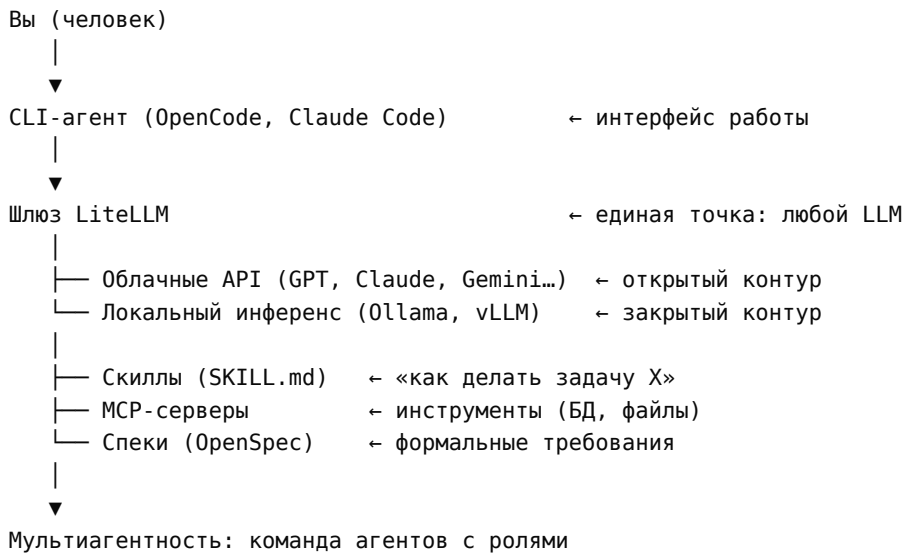
Docker-стек — полигон для практики

Postgres + pgvector	Production-grade векторное хранилище для RAG
MinIO	S3-совместимое хранилище документов
Jupyter	Лаборатория экспериментов
Spark	Опционально — ETL-пайплайны индексации

Не обязательно использовать весь стек — это доступные ресурсы.

3. Большая картина

Современный «агентный стек», который мы будем собирать по слоям на протяжении всего курса:



> **Ключевая мысль курса.** LLM — это только мозг. **Агент** = **LLM** + **инструменты** + **цикл действий.** Всё остальное (скиллы, спеки, MCP, шлюз) — способы дать этому мозгу знания, руки и сменные «движки».

4. Ключевые принципы курса

1. **Provider-agnostic всё.** Нигде не хардкодим конкретную модель. OpenAI-совместимый API как универсальный контракт + LiteLLM как шлюз с роутингом и fallback. Один и тот же код работает в открытом и закрытом контуре — меняется только конфигурация.
2. **Локально-первый подход.** Фундамент строим на локальных моделях: это must-have для закрытого контура. Облако добавляем как сравнение и как «тяжёлую артиллерию».
3. **Руками, потом фреймворки.** Сначала пишем сами (агентный цикл на 50 строк, RAG без «чёрных ящиков»), потом сравниваем с готовыми инструментами — так появляется понимание, а не магия.
4. **Практика на своём стеке.** Docker-стек (pgvector, MinIO, Jupyter) — наш полигон. Артефакты курса сразу пригодны для демонстрации на работе.
5. **Оценка качества — не опция.** RAG и агенты без метрик — игрушка. Учимся измерять с первого серьёзного этапа.

5. Этапы обучения

Девять этапов: от фундамента до продакшна. Каждый заканчивается работающим артефактом, а не конспектом. Ориентировочная длительность указана при темпе 5–10 часов в неделю.

Этап 0. Фундамент LLM и API

~ 1 неделя · база

Зачем: понять, что происходит «под капотом» любого вызова модели. Без этого всё остальное — магия.

Что изучаем

- **Токены:** как модель видит текст; почему «строчная буква» и «пробел» влияют на стоимость и качество.
- **Контекстное окно:** что помещается, что «забывается», стратегии работы с длинными текстами.
- **Параметры генерации:** температура, top-p, max tokens — на примерах, где каждая настройка ломает или чинит ответ.
- **Роли сообщений:** system / user / assistant; system prompt как «должностная инструкция» модели.
- **OpenAI-совместимый API** как единый контракт: один код — любой провайдер, облачный или локальный. Меняется только base_url и ключ.

> **Нюанс, который новички упускают.** Чат в браузере — готовый продукт. API — это уровень, на котором *вы* становитесь разработчиком продукта: управляете промптом, историей, инструментами. Именно здесь строятся все агенты.

Практика

1. Скрипт: один вопрос — один ответ через API.
2. Диалог с памятью (передача истории сообщений) — зачаток агента.
3. Эксперимент: один и тот же вопрос с температурой 0, 0.7 и 1.2 — сравнить ответы.

> **Результат этапа.** Рабочий скрипт-диалог; понимание токенов, контекста и ролей; уверенность, что «подключить LLM через API» — это 15 строк кода.

Этап 1. Локальный инференс-стек

~ 2 недели · закрытый контур · фундамент

Зачем: это фундамент вашего требования «закрытый контур». Всё последующее обучение будет работать на этом стеке.

Что изучаем

- **Ollama** — быстрый старт локальных моделей: одна команда — работающая модель.
- **vLLM и llama.cpp** — production-инференс: батчинг, управление VRAM, OpenAI-совместимый сервер.
- **Квантизация:** GGUF, Q4/Q5/Q6, MXFP4. Что влезает в 16 GB VRAM и чем жертвуем. Правило большого пальца: вес модели в квантизации + запас под контекст (KV-кэш).
- **Зоопарк моделей и их роли:** генеральная (Hermes, Qwen) для диалогов; кодовая — для coding-агентов; эмбеддинговая (**многоязычная!** — критично для русского) — для RAG; reranker — для переранжирования поиска.
- **LiteLLM как шлюз:** единый endpoint, за которым — облако и локалка. Роутинг по задачам (дешёвая локальная для простого, мощная облачная для сложного), fallback при сбоях. Паттерн, напрямую переносимый в корпоративный контур.
- Развёртывание в Docker-стеке: инференс-сервер как ещё один контейнер рядом с Postgres и MinIO.

> **Нюанс.** Выбор модели под задачу — отдельный навык. С этого этапа ведём «дневник наблюдений»: какая модель на каких задачах спотыкается.

Практика

1. Поднять Ollama, скачать модель ~14B, замерить скорость (токенов/сек) на вашей 5060 Ti.
2. Поднять инференс-сервер в Docker, проверить OpenAI-совместимый endpoint из Jupyter.
3. Настроить LiteLLM: один адрес две модели (локальная + облачная), переключение строкой в конфиге.

> **Результат этапа.** Собственный инференс-стек в Docker; шлюз LiteLLM с локальной и облачной моделью; понимание квантизации и бюджета VRAM.

Этап 2. CLI и агентное окружение на Windows

~ 1 неделя · инструменты

Зачем: агенту нужно *выполнять действия* — читать файлы, запускать команды, работать с git. Поэтому самые мощные инструменты — терминальные. Готовим правильное окружение.

Что изучаем

- **WSL2** — правильный дом для агентов на Windows: Linux-окружение с доступом к вашим файлам и GPU.

- **OpenCode** — открытый терминальный coding-агент: режимы **Plan** (только исследует и предлагает) и **Build** (реально правит файлы), переключение моделей командой /model.
- Подключение OpenCode к **вашим локальным моделям** через шлюз — сразу приучаемся к закрытому контуру.
- **AGENTS.md** — файл-инструкция о проекте: как рассказать агенту о структуре кода, конвенциях и правилах.

Практика

1. Установить WSL2 и OpenCode; настроить провайдера через LiteLLM.
2. На тестовом репозитории: /init обсудить задачу в Plan реализовать в Build.
3. Написать свой первый AGENTS.md.

> **Результат этапа.** Рабочее агентное окружение; OpenCode, работающий на локальной модели через ваш шлюз.

Этап 3. Агенты изнутри

~ 2 недели · ключевая практика

Зачем: понять агента на уровне кода. Когда вы напишете агентный цикл сами — OpenCode и Claude Code перестанут быть магией: они устроены ровно так, только с тысячей улучшений.

Что изучаем

- **Агентный цикл:** восприятие решение действие наблюдение результата. Решения принимает LLM, действия выполняются через инструменты.
- **Function calling / tool use:** как модель «просит» вызвать функцию; схемы инструментов (JSON Schema); обработка отказов и ошибок.
- Управление контекстом: что класть в историю, когда подрезать.
- Prompt engineering на уровне систем: как формулировать задачи, чтобы агент не «сходил с рельсов».

Практика

1. Свой мини-агент на 50–100 строк против **локальной модели**: читает файлы, выполняет SQL против вашего Postgres, ведёт диалог, пока задача не решена.

```
# Скелет агентного цикла
while not done:
    response = llm(messages, tools=[read_file, run_sql])
    if response.wants_tool_call:
        result = execute_tool(response.tool_call)  # агент «действует»
        messages.append(result)                    # агент «наблюдает»
    else:
        done = True                                # агент закончил
```

> **Результат этапа.** Работающий собственный агент; полное понимание function calling — база для MCP и мультиагентности.

Этап 4. RAG с нуля на локальных моделях

~ 3 недели · ключевой этап · сквозной проект

Зачем: это ваш сквозной проект и главная цель курса. Строим всё руками, без фреймворков-«чёрных ящиков», затем сравниваем с фреймворками.

Что изучаем

- **Эмбеддинги:** векторное представление текста; многоязычные модели (критично для русского!); размерность и её влияние на скорость и память.
- **Чанкинг:** стратегии нарезки документов (фиксированный, по абзацам, семантический, с перекрытием) и их влияние на качество ответов.
- **pgvector:** тип vector, индекс HNSW, запросы по косинусной близости, метаданные и фильтры («только регламенты 2025 года»).
- **Гибридный поиск:** вектор + полнотекстовый BM25 в том же Postgres; слияние результатов (RRF).
- **Реранкинг:** отдельная модель переранжирует топ-кандидатов — простой способ сильно поднять качество.
- **Сборка ответа:** как подкладывать найденное в промпт; цитирование источников; борьба с галлюцинациями.
- **Оценка качества RAG:** метрики поиска (hit rate, MRR) и ответа; тестовый набор вопросов; регрессионные проверки. То, что новички пропускают, а в проде без этого никак.

Архитектура сквозного проекта

```
Документы в MinIO
      │
      ▼
Пайплайн индексации (Jupyter → скрипт):
  чанкинг → локальные эмбеддинги → запись в pgvector
      │
      ▼
Запрос пользователя:
  эмбеддинг запроса → гибридный поиск (вектор + BM25)
  → реранкинг → топ-N фрагментов
  → LLM (любая, через LiteLLM) → ответ с цитатами
```

Практика

1. Минимальный RAG на 100 строк в Jupyter: эмбеддинги pgvector поиск ответ.
2. Эксперименты с чанкингом на реальных документах: меняем стратегию — меряем качество.

3. Гибридный поиск и реранкинг: замер прироста качества на тестовом наборе вопросов.
4. Переключение генерации между локальной и облачной моделью через LiteLLM — сравнение ответов на одних и тех же найденных фрагментах.

> **Результат этапа.** Полноценная RAG-система на вашем стеке (MinIO + pgvector + локальные модели), с оценкой качества и провайдеро-независимой генерацией. Артефакт, который можно показать на работе.

Этап 5. Скиллы и MCP

~ 1–2 недели · расширение агента

Зачем: научить агента «профессиям» и подключить к нему внешние инструменты стандартным способом.

Что изучаем

- **Скиллы (SKILL.md):** упакованная инструкция, которую агент подгружает, когда задача попадает в её домен. Аналогия: LLM — выпускник вуза с общим образованием, скилл — должностная инструкция + чек-листы конкретной работы. Структура: название, описание-триггер, пошаговые инструкции, ресурсы.
- **MCP (Model Context Protocol):** открытый протокол подключения инструментов — «USB-C для агентов». MCP-сервер пишется один раз — работает с любым совместимым агентом.
- Локальные MCP-серверы: Postgres MCP (запросы к БД), filesystem — всё внутри контура, без облака.

Практика

1. Написать свой скилл (например, «как оформлять аналитические отчёты») и посмотреть, как меняется поведение агента.
2. Поднять Postgres MCP-сервер, подключить к OpenCode; дать задачу, требующую живого SQL.

> **Результат этапа.** Собственный скилл; агент с подключёнными локальными MCP-инструментами.

Этап 6. Spec-driven development

~ 1 неделя · методология

Зачем: дисциплина разработки через спецификации. Вместо «написал промпт получил код переделываешь 10 раз»: сначала согласовываем с агентом формальную спецификацию, код пишется по ней.

Что изучаем

- **OpenSpec** — открытый CLI-фреймворк, работающий с любым AI-ассистентом.
- Рабочий цикл: proposal (что и зачем) ревью apply (реализация по согласованной спеке) archive (спека становится живой документацией).
- Когда SDD окупается (средние и большие задачи, где требования легко потерять) и когда это лишняя бюрократия (маленькие задачи — хватит хорошего промпта).

Практика

1. Одна реальная фича в тестовом проекте через полный цикл OpenSpec.

> **Результат этапа.** Понимание spec-driven подхода и границ его применимости.

Этап 7. Мультиагентные системы

~ 2–3 недели · кейсы дата-инженера

Зачем: один агент с длинной задачей «забывает» контекст и теряет качество. Команда специализированных агентов — короче контекст у каждого, чётче ответственность, параллелизм.

Что изучаем

- Паттерны: **оркестратор** + **воркеры**, pipeline, debate/ревью (один агент пишет, другой критикует).
- Мультимодельное ревью: критика одной модели другой снижает систематические ошибки.
- **LangGraph** (и обзор альтернатив: CrewAI, OpenAI Agents SDK) — построение агентных систем в коде.
- Субагенты и параллельная разработка (git worktrees).

Практика

1. **Дуэт «писатель + критик»** через API (~100 строк) — концептуальная база мультиагентности.
2. **Text-to-SQL агент:** вопрос на естественном языке SQL результат; с проверкой запроса перед выполнением.
3. **Агент качества данных:** автоматические проверки, поиск аномалий, отчёт.
4. **Агент-документатор витрин** (по желанию).

> **Результат этапа.** 2–3 рабочих агентных кейса под вашу профессию; понимание оркестрации.

Этап 8. Закрытый контур и продакшн

~ 1–2 недели · финальный проект

Зачем: то, что отличает «поигрался дома» от «внедрил на работе».

Что изучаем

- **Air-gap развёртывание:** зеркала моделей, офлайн-зависимости (pip/npm зеркала), перенос Docker-образов.
- **Безопасность:** prompt injection, доступ агента к данным (минимально необходимые права), секреты и ключи, аудит действий агента.
- **Оценка и мониторинг агентов:** трассировка шагов, метрики качества и стоимости, алерты на деградацию.
- **Реестр моделей:** версии моделей как версии кода; откат при деградации.

Практика

1. **Финальный проект:** полностью локальная агентная система на вашем Docker-стеке — RAG + агент с инструментами + LiteLLM-шлюз, развёрнутая так, как это было бы в закрытом контуре на работе.

> **Результат этапа.** Портфолио-система и чек-лист внедрения для рабочего контура.

6. Must-have стек на конец 2026 года

Категория	Инструменты
CLI-агент	OpenCode (открытый, гибкий) и/или Claude Code
Шлюз к моделям	**LiteLLM** — сердце provider-agnostic архитектуры
Облачный доступ	API-ключ провайдера + OpenRouter как агрегатор
Локальный инференс	Ollama (старт), vLLM (продакшн)
Векторное хранилище	pgvector — уже есть в вашем стеке
Расширение агента	MCP-серверы + собственные скиллы
Методология	AGENTS.md в каждом проекте, OpenSpec для больших фич
Оркестрация агентов	LangGraph
База	git, терминал (WSL2), Python

7. Карточка терминов

Термин	Одной фразой
LLM	Большая языковая модель — «мозг», предсказывающий следующий токен
Токен	Кусочек текста (~¾ слова), единица обработки и оплаты
Контекстное окно	Сколько токенов модель «видит» одновременно
API	Программный интерфейс к модели; OpenAI-совместимый API — единый стандарт общения
CLI	Интерфейс командной строки; среда, где живут терминальные агенты
Агент	LLM + инструменты + цикл «решение действие наблюдение»
Function calling	Способ модели «попросить» вызвать функцию (инструмент)
RAG	Поиск по документам + генерация ответа по найденному
Эмбеддинг	Векторное представление текста для семантического поиска
Квантизация	Сжатие весов модели (Q4/Q6…) ради памяти и скорости ценой капли качества
Скилл (SKILL.md)	Упакованная инструкция, обучающая агента классу задач
MCP	Открытый протокол подключения инструментов к агентам («USB-C»)
OpenSpec	Фреймворк разработки через спецификации (spec-driven development)
Мультиагентность	Команда специализированных агентов с оркестратором
LiteLLM	Шлюз-прокси: один endpoint — любые модели, роутинг и fallback
Закрытый контур	Среда без доступа к внешним сервисам; всё локально

8. Формат работы

1. **Один этап за раз.** Теория порциями + мини-практика на своей машине.
2. **Домашние артефакты.** Каждый этап заканчивается работающим результатом (скрипт, сервис, агент) — не конспектом.
3. **Дневник наблюдений.** Фиксируем, какие модели на каких задачах спотыкаются — ваша личная база интуиции по выбору моделей.
4. **Вопросы — в любой момент.** Останавливаемся и разбираем подробно, не идём дальше с пробелами.

План хранится и в памяти проекта: в любой сессии продолжим с того места, где остановились.